

Porte-Monnaie

Dossier complet du projet

Application de bureau JavaFX pour gerer son budget personnel



Objectif du projet

Créer une application simple, moderne et utilisable sur PC pour suivre ses finances personnelles : revenus, dépenses, solde, épargne, statistiques et exports.

Java

JavaFX

SQLite

JDBC

GitHub

PDF

Livrables

- Application Windows avec exécutable.
- Code source versionné sur GitHub.
- Documentation claire pour comprendre et présenter le projet.
- Support visuel type webtoon intégré dans ce dossier.

Sommaire

01 **Contexte et objectif**
Pourquoi cette application existe.

02 **Fonctionnalites**
Ce que l'utilisateur peut faire.

03 **Parcours utilisateur**
De la connexion a l'export.

04 **Architecture**
Comment le code est organise.

05 **Base de donnees**
Les tables principales.

06 **Securite**
Mot de passe et injections SQL.

07 **Explication du code**
Les classes importantes.

08 **Tests et executable**
Ce qui a ete verifie.

09 **Demo BTS**
Scenario pour le jury.

10 **Recap webtoon**
Version visuelle et rapide.

1. Contexte et objectif

Probleme de depart

- Beaucoup de personnes suivent leurs finances avec plusieurs outils differents.
- Les revenus, depenses et projets d'epargne ne sont pas toujours centralises.
- Il est difficile de savoir rapidement combien il reste et ou part l'argent.

Solution proposee

Porte-Monnaie est une application locale qui rassemble les informations financieres importantes dans une interface style portefeuille numerique.

L'objectif est de proposer un outil simple a utiliser, mais assez complet pour demontrer les competences attendues en BTS SIO SLAM.

Choix important

La version actuelle utilise SQLite. C'est coherent pour une application de bureau locale, car aucun serveur n'est necessaire. Une migration MySQL reste possible en evolution.

2. Fonctionnalités

Compte utilisateur

- Creation de compte
- Connexion email + mot de passe
- Nom utilisateur affiche

Tableau de bord

- Solde actuel
- Revenus et depenses
- Epargne suivie
- Dernieres transactions

Transactions

- Ajout
- Modification
- Suppression
- Filtres par date, categorie, montant

Epargne

- Objectifs
- Montant cible
- Progression
- Reste a economiser

Statistiques

- Repartition par categorie
- Evolution du solde
- Mois le plus depensier

Exports

- CSV
- PDF
- Donnees locales persistantes

Ces fonctionnalites permettent une demonstration complete : compte, operations, graphiques, objectif d'epargne et export.

3.

1

Connexion

L'utilisateur ouvre l'application et se connecte.

2

Tableau de bord

Il visualise son solde, ses revenus, ses dépenses et son épargne.

3

Ajout

Il ajoute un revenu ou une dépense avec date, catégorie et description.

4

Analyse

Il consulte l'historique, les filtres et les statistiques.

5

Épargne

Il crée un objectif et suit sa progression.

6

Export

Il exporte les transactions en CSV ou PDF.

4. Architecture

Le projet est organise en couches. Chaque partie a un role clair, ce qui rend le code plus facile a comprendre et a modifier.

ui

Pages JavaFX : affichage, boutons, formulaires.

service

Logique metier : calculs, authentication, exports.

dao

Acces aux donnees : requetes SQL preparees.

model

Objets metier : User, Transaction, Goal, Category.

db

Connexion SQLite et creation des tables.

**Phrase simple a dire : l'interface ne parle pas directement avec la base.
Elle passe par les services, qui utilisent ensuite les DAO.**

5. Base de donnees

users

Compte utilisateur : nom, email, hash, sel, date de creation.

transactions

Operations financieres : type, montant, categorie, date, description.

categories

Categories par default et categories personnalisees.

goals

Objectifs d'epargne : titre, cible, montant actuel.

settings

Petits reglages : nom affiche, devise.

Pourquoi SQLite ?

SQLite est integre dans l'application et stocke les donnees dans un fichier local. C'est plus simple pour une application PC. MySQL peut etre ajoute plus tard si l'application devient multi-utilisateur ou connectee a un serveur.

6. Securite

Mot de passe

- Le mot de passe n'est jamais stocke en clair.
- Un sel aleatoire est genere pour chaque compte.
- Le mot de passe et le sel sont hashes avec SHA-256.
- A la connexion, l'application recalcule le hash et compare le resultat.

Injection SQL

Les requetes importantes utilisent PreparedStatement. Les valeurs saisies par l'utilisateur ne sont pas collees directement dans le SQL.

C'est une protection importante contre les injections SQL.

Limite a expliquer : SHA-256 est acceptable pour montrer le principe dans un projet BTS, mais dans une application professionnelle on prefererait BCrypt, Argon2 ou PBKDF2.

7. Explication du code

Transaction

Représente une entrée ou une sortie d'argent. Le montant reste positif, le type indique revenu ou dépense.

WalletService

Calcule les totaux, le solde, les statistiques et appelle les DAO.

AuthService

Gère l'inscription, la connexion et le hashage du mot de passe.

TransactionDao

Exécute les requêtes SQL pour ajouter, modifier, supprimer et filtrer les transactions.

GoalsPage

Affiche les objectifs d'épargne et permet d'ajouter une somme.

styles.css

Contient le thème sombre, cuir et doré de l'application.

8. Tests et executable

Ce qui a ete verifie

- Compilation Maven du projet.
- Creation d'un compte.
- Connexion avec un bon mot de passe.
- Refus d'un mauvais mot de passe.
- Ajout d'un revenu et d'une depense.
- Calcul du solde.
- Objectifs d'epargne.
- Demarrage de l'executable Windows.

Fichiers utiles

- Executable : dist/Porte-Monnaie/Porte-Monnaie.exe
- Zip complet : Porte-Monnaie-Windows.zip
- Explication : EXPLICATION_CODE_BTS.md
- PDF final : output/pdf/porte-monnaie-dossier-complet.pdf

Important : pour deplacer l'application, il faut garder le dossier complet, pas seulement le fichier .exe, car le runtime Java est inclus a cote.

9. Demo BTS

- 01** Ouvrir l'application depuis le raccourci bureau.
- 02** Créer un compte ou se connecter.
- 03** Ajouter un revenu, par exemple un salaire.
- 04** Ajouter une dépense, par exemple des courses.
- 05** Revenir au tableau de bord pour voir le solde.
- 06** Créer un objectif d'épargne, par exemple Acheter un PC.
- 07** Afficher les statistiques.
- 08** Exporter les transactions en CSV ou PDF.
- 09** Montrer rapidement la base de données et le code.

Phrase cle : ce projet montre du Java, JavaFX, JDBC, une base de données, de la sécurité et GitHub.

10. Recap webtoon

Le probleme

Je veux suivre mon argent sans ouvrir trois applications differentes.

La reponse

Un portefeuille numerique local : revenus, depenses, epargne, statistiques et exports.



La conclusion

Porte-Monnaie est un projet complet pour un BTS SIO SLAM : application JavaFX, architecture en couches, base SQLite, securite et gestion de projet GitHub.

Conclusion

Ce que le projet demontre

- Developpement logiciel en Java.
- Interface graphique JavaFX.
- Architecture MVC simplifiee / couches separees.
- JDBC et base de donnees relationnelle.
- Securite de base : hashage et requetes preparees.
- Versionnement GitHub et documentation.

Evolutions possibles

- Migration vers MySQL.
- Plusieurs comptes bancaires.
- Export Excel XLSX.
- Abonnements recurrents.
- Notifications de budget.
- Application mobile.

Phrase finale : l'application est utilisable sur PC et couvre les competences importantes attendues pour l'epreuve professionnelle.